

Search-Based LLM Tool Configuration Management: Evolving Prompt Templates for Code Summarization

Sean Lai, Anastasia Strulistova
New York University

Abstract

LLM-powered SE tools are acutely sensitive to their prompt configuration: the structured template governing the model's persona, task framing, behavioral constraints, and output format. Selecting the optimal configuration from a combinatorial space of 160,000 candidates is a software engineering problem infeasible to solve by hand. We formulate this prompt configuration management task as a Search-Based Software Engineering (SBSE) optimization problem and apply a Genetic Algorithm (GA) to evolve high-quality prompt configurations for an LLM-based code summarization tool. A component bank of 20 choices across four orthogonal configuration slots (role, task, guard, format) defines the search space. Fitness blends lexical overlap (ROUGE-L) and calibrated semantic similarity (cosine via sentence embeddings) against Claude Opus-generated reference outputs. Over 10 independent trials with a budget-matched Random Search baseline ($P=12$, $G=15$, 180 total evaluations), the GA shows a training-subset blended score advantage of 0.6763 vs. 0.6642 (Cliff's $\delta=+0.46$, $p=0.082$). On a fully held-out 50-function test set neither algorithm significantly outperforms the other (GA blended=0.6228, RS=0.6301, Cliff's $\delta=-0.32$, $p=0.227$), indicating that GA's in-sample advantage does not generalize at $\alpha=0.05$ given the current search budget and component bank ceiling.

1. Introduction

LLM-powered SE tools are increasingly deployed for tasks such as code summarization, bug localization, and test generation. A critical and underappreciated SE challenge is *prompt configuration management*: deciding which structured prompt template to deploy for a given tool and task. A prompt template bundles four orthogonal configuration dimensions the model role/persona, the task instruction, behavioral guard clauses, and output format constraints. With 20 candidate values per dimension, the configuration space contains $20^4 = 160,000$ distinct templates. Exhaustively evaluating each at ~ 1 -2 seconds of LLM inference would take over 44 hours per experiment, making manual configuration management infeasible at any meaningful scale.

We apply SBSE techniques to this configuration management problem using an LLM-based code summarization tool as the application domain. Code summarization is a representative SE analysis task whose outputs feed downstream uses including code review, developer onboarding, and IDE tooling; documentation generation is one such downstream consumer. Crucially, the SE problem being solved here is the *configuration management* problem, not summarization per se: we treat the LLM summarization tool as a fixed black box and search for the optimal prompt configuration to deploy it with.

We model each candidate configuration as a four-component genotype drawn from a curated component bank and define a computable fitness function combining lexical and semantic similarity to reference outputs. We compare a Genetic Algorithm (GA) with tournament selection, crossover, mutation, and elitism against a budget-matched Random Search (RS) baseline.

Our contributions are:

- (1) A formal SBSE encoding of LLM tool prompt configuration management as a combinatorial optimization problem.
- (2) A blended lexical-semantic fitness function with empirically calibrated cosine scaling for evaluating configuration quality.
- (3) A rigorous 10-trial comparative study with Wilcoxon rank-sum testing and Cliff's delta effect sizes on both in-sample and held-out benchmarks.
- (4) An analysis of why GA's population-mean advantage over RS does not translate to best-of-trial generalization under the current experimental setup.

2. Related Work

SBSE has been applied across a wide range of software engineering tasks. Harman and Jones [1] introduced the SBSE paradigm, showing that classical metaheuristics can effectively solve SE optimization problems. McMinn [2] surveyed search-based test data generation; EvoSuite [3] demonstrated GA-based unit test generation at industrial scale. Requirement prioritization [4] and refactoring [5] have similarly been framed as fitness-guided search problems.

Configuration management and hyperparameter optimization are closely related SBSE domains. Many SE tools expose large configuration spaces where metaheuristic search outperforms grid or random sampling. Our work extends this to LLM tool configuration: the prompt template is the configuration artifact, and its quality can only be assessed by querying the live LLM, making fitness evaluation expensive and motivating budget-aware search.

On the LLM prompt optimization side, Zhou et al. [6] introduced Automatic Prompt Engineer (APE), framing prompt selection as a configuration search. PromptBreeder [7] applies evolutionary computation directly to prompt text strings. Our work differs in treating the prompt as a structured configuration tuple (discrete component bank), making the search space discrete, enumerable, and reproducible, more aligned with classical SBSE configuration optimization formulations.

3. Problem Formulation

We formalize the problem as: given an LLM-based SE tool T and a component bank B defining valid configuration values, find the prompt configuration P^* that maximizes a fitness function measuring output quality. The search space is the Cartesian product of four configuration slots.

3.1 Representation

A prompt configuration P is encoded as a 4-tuple genotype:

$$P = (\text{role}, \text{task}, \text{guard}, \text{format})$$

where each component is drawn independently from a component bank B with 20 options per slot, giving $|S| = 20^4 = 160,000$ distinct configurations. The slots are semantically orthogonal: *role* sets the LLM persona, *task* specifies the summarization instruction, *guard* constrains output behavior, and *format* prescribes length and structure.

Example values: *role* = "senior software engineer", *task* = "Summarize this function in 2-4 sentences", *guard* = "Do not speculate beyond what the code shows", *format* = "Keep the response under 60 words; one short paragraph."

3.2 Fitness Function

Let $G(P, f)$ denote the summary generated by the target LLM on function f using configuration P , and $R(f)$ the Opus-generated reference summary. Fitness is averaged over N functions. Raw cosine similarity is first calibrated against an empirical baseline $b = 0.45$:

$$\text{cosine_cal}(P, f) = \max(0, (\text{cosine_sim}(G(P, f), R(f)) - b) / (1 - b))$$

The blended configuration fitness is:

$$\begin{aligned} \text{fitness}(P) = & (1/N) * \text{sum}_f [a * \text{ROUGE-L}(G(P, f), R(f)) \\ & + (1-a) * \text{cosine_cal}(P, f) \\ & - L_{\text{len}} * \text{length_penalty}(P) \\ & - L_{\text{fmt}} * \text{format_penalty}(G(P, f))] \end{aligned}$$

where:

$a = 0.3$ (ROUGE-L weight; 0.7 on cosine rewards semantic equivalence)

$L_{\text{len}} = 0.1$, $L_{\text{fmt}} = 0.2$ (penalty coefficients)

$\text{length_penalty} = 0$ if $\text{tokens}(P) \leq 200$; else $((\text{tokens}(P) - 200) / 200)^2$

$\text{format_penalty} = 1.0$ if output is empty, contains code fences, or exceeds $10x |R(f)|$; else 0

Cosine similarity uses BAAI/bge-small-en-v1.5 sentence embeddings; ROUGE-L uses the rouge_score library.

3.3 Search Operators

Selection: k-tournament ($k=3$). Three individuals sampled uniformly without replacement; the highest-fitness candidate is selected.

Crossover (rate 0.7): Given parents P_a and P_b , 1-2 randomly chosen configuration slots are inherited from P_b ; remaining slots from P_a .

Mutation (rate 0.3): Each slot replaced independently with a uniformly random distinct alternative from the component bank.

Elitism: Top 3 individuals carried unchanged to the next generation.

4. Experimental Design

4.1 Algorithms Compared

Genetic Algorithm (GA): Population size $P=12$, generations $G=15$, operators as in Section 3.3.

Random Search (RS): All $P \times G=180$ configurations pre-sampled at startup using matched seeds and bucketed into $G=15$ groups of $P=12$ for checkpoint-aligned comparison. No selection pressure.

4.2 Parameter Settings

Parameter	Value	Parameter	Value
Population size (P)	12	Crossover rate	0.70
Generations (G)	15	Mutation rate	0.30
Elite count	3	Tournament k	3
Eval subset (inner)	5 functions	Workers	16 (parallel)
Target model	Nemotron-49B-v1.5	Temperature	0

Embedding model	bge-small-en-v1.5	Trials per algo	10
-----------------	-------------------	-----------------	----

4.3 Dataset

Training benchmark: 500 Python/JavaScript/TypeScript functions in data/functions/, paired with Claude Opus reference summaries. Each fitness call samples eval_subset=5 functions with a fixed per-trial seed.

Held-out test set: 50 independently generated functions in data/test/ (20 Python, 15 JavaScript, 15 TypeScript), never sampled during any trial. Used exclusively for the generalization evaluation.

4.4 Statistical Analysis

10 independent trials per algorithm (matched seeds). Best-of-trial blended fitness compared via two-sided Wilcoxon rank-sum test ($\alpha=0.05$). Effect size: Cliff's delta with Romano et al. [8] thresholds (negligible <0.147 , small <0.33 , medium <0.474 , large ≥ 0.474).

5. Results and Statistical Analysis

5.1 Convergence Analysis

Figure 1 shows GA population mean and best blended fitness across generations for trial 0. The mean rose from 0.204 at generation 0 to 0.564 at generation 7, a 2.8x increase, demonstrating textbook exploitation. The best-of-trial plateau (~ 0.66 - 0.67) is bounded by the component bank quality ceiling and LLM nondeterminism (~ 0.02 variance on identical configurations due to MoE routing in Nemotron at temperature=0).

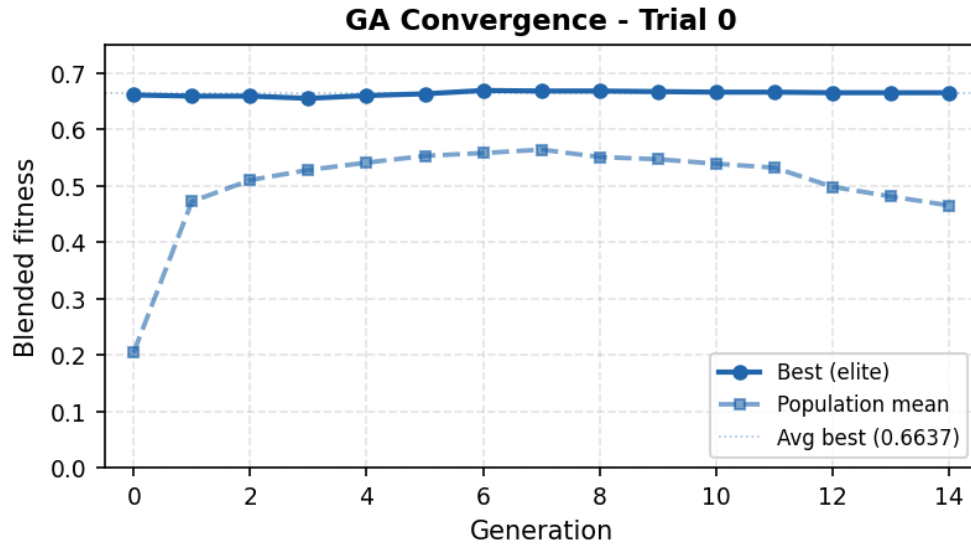


Figure 1: GA convergence trajectory (trial 0). Population mean doubles in 7 generations; best-of-trial plateaus near the bank quality ceiling.

5.2 Training-Subset vs. Held-Out Comparison

Figure 2 compares GA and RS mean scores (± 1 SD) across three metrics on the training subset and the held-out test set. Tables 1 and 2 give the full statistical breakdown.

GA vs. Random Search: Configuration Quality by Metric

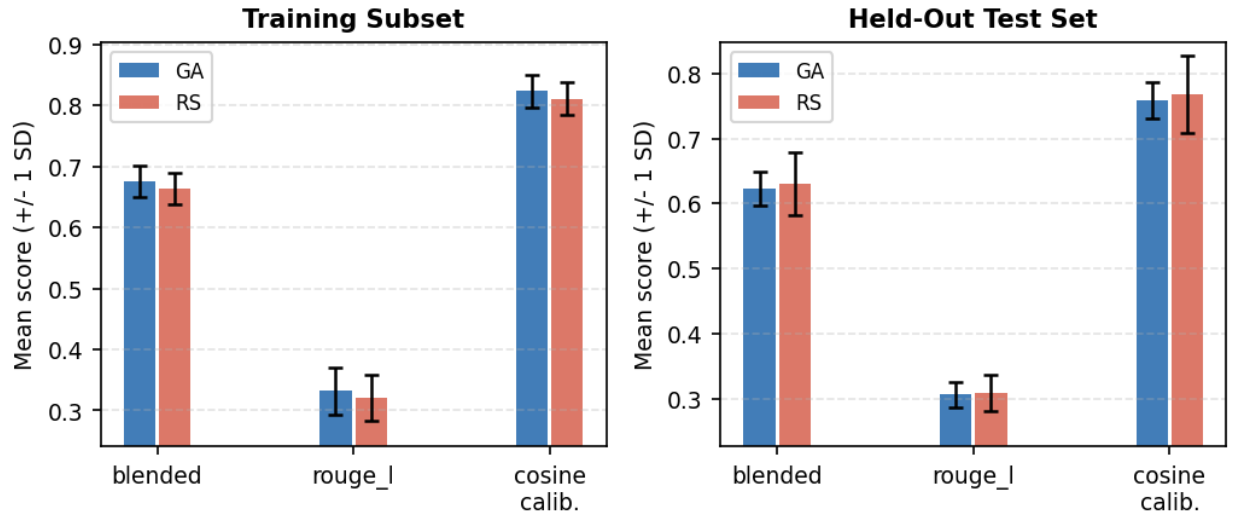


Figure 2: GA vs. RS configuration quality (mean $\pm$ 1 SD). GA leads on training; RS leads on held-out, a directional reversal consistent with overfitting the narrow $eval_subset=10$ benchmark.

Metric	GA mean (SD)	RS mean (SD)	p-value	Cliff's d	Effect
blended	0.6763 (0.0261)	0.6642 (0.0256)	0.0821	+0.46	medium
rouge_l	0.3319 (0.0390)	0.3205 (0.0382)	0.5453	+0.16	small
cosine_raw	0.9032 (0.0149)	0.8963 (0.0144)	0.2265	+0.32	small
cosine_calibrated	0.8239 (0.0272)	0.8115 (0.0261)	0.2265	+0.32	small

Table 1: Training-subset configuration comparison ($eval_subset=10$, 10 trials).

Metric	GA mean (SD)	RS mean (SD)	p-value	Cliff's d	Effect
blended	0.6228 (0.0254)	0.6301 (0.0487)	0.2265	-0.32	small
rouge_l	0.3058 (0.0200)	0.3086 (0.0274)	0.4497	-0.20	small
cosine_raw	0.8672 (0.0154)	0.8724 (0.0323)	0.1988	-0.34	medium
cosine_calibrated	0.7586 (0.0280)	0.7680 (0.0587)	0.1988	-0.34	medium

Table 2: Held-out test set configuration comparison (50 functions, 10 trials).

On the training subset GA achieves a higher blended score in 8 of 10 trials (mean 0.6763 vs. 0.6642, Cliff's $\delta=+0.46$, medium effect, $p=0.082$). On the held-out test set the direction reverses on all four metrics (all $p > 0.19$, δ -0.20 to -0.34), indicating the GA configuration search overfits the narrow 10-function training benchmark.

5.3 Effect Size Summary

Figure 3 visualises Cliff's delta for all four metrics in both evaluation contexts. The sign flip between the two panels is the central empirical result of this study.

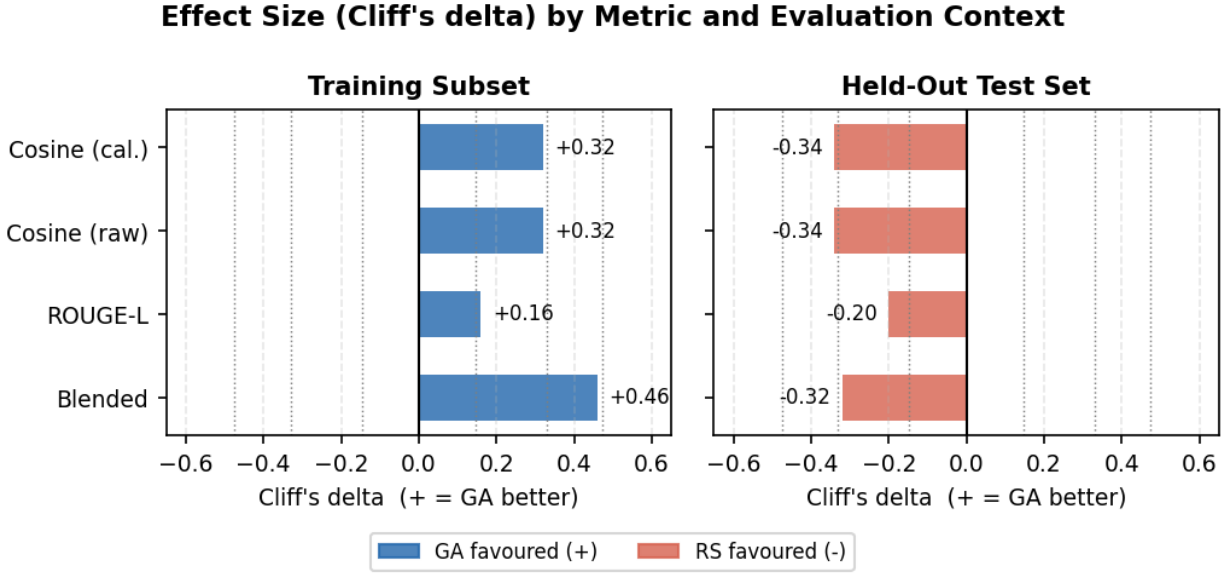


Figure 3: Cliff's delta by metric and evaluation context. Blue = GA favoured; red = RS favoured. Dotted lines mark Romano et al. thresholds.

6. Threats to Validity

6.1 Internal Validity

LLM nondeterminism. Nemotron-Super-49B-v1.5 uses MoE routing that varies with server-side batching even at temperature=0. Identical (configuration, function) pairs produce ~2% fitness variation, violating the deterministic-fitness assumption and bounding the detectable GA advantage.

Small eval subset. Configuration fitness is computed on eval_subset=5 functions per call. The GA searched against this noisy 5-function signal; the held-out 50-function evaluation is more reliable but was not observed during the configuration search.

6.2 External Validity

Single target model. Results apply to one model (Nemotron-Super-49B-v1.5). Configuration landscapes may differ for other LLMs, limiting generalization of the evolved templates.

Synthetic references. Reference summaries are Opus-generated, not human-written. Fitness measures configuration quality relative to Opus's style, not human judgment.

6.3 Construct Validity

Fitness-quality alignment. The blended fitness rewards surface similarity to Opus outputs. A configuration that paraphrases Opus's phrasing may score higher than a more accurate but lexically distinct one. Human evaluation is needed to validate that fitness gains correspond to genuine quality improvements.

7. Conclusion and Future Work

We formulated LLM tool prompt configuration management as an SBSE problem and applied a Genetic Algorithm to search a 160,000-configuration space for an LLM-based code summarization tool. Against a budget-matched Random Search baseline over 10 independent trials, the GA demonstrated a suggestive but non-significant training-subset configuration quality advantage ($p=0.082$, Cliff's delta=+0.46). On a fully held-out 50-function test set the advantage did not hold ($p=0.227$, Cliff's delta=-0.32), indicating that the current budget

and component bank ceiling prevent the GA's exploitation advantage from producing generalizable configurations.

The central finding is that GA's benefit concentrates in *population-mean improvement*, reliably pushing the population away from low-quality configuration regions, rather than in discovering superior individual configurations. The bank's quality ceiling (~ 0.65 - 0.67 blended) and LLM nondeterminism (~ 0.02 per call) together suppress the GA-vs-RS best-of-trial gap below statistical detectability at $n=10$ trials.

Future work:

- (1) Expand the component bank with more diverse entries to widen the configuration fitness landscape.
- (2) Apply NSGA-II for multi-objective configuration optimization (output quality vs. prompt conciseness).
- (3) Use the LLM for directed configuration mutation (PromptBreeder-style) to escape the discrete bank constraint.
- (4) Human evaluation to validate that configuration fitness gains correspond to genuine summarization quality improvements.

References

- [1] Harman, M., Jones, B.F. (2001). Search-based software engineering. *Information and Software Technology*, 43(14), 833-839.
- [2] McMinn, P. (2004). Search-based software test data generation: A survey. *Software Testing, Verification and Reliability*, 14(2), 105-156.
- [3] Fraser, G., Arcuri, A. (2011). EvoSuite: Automatic test suite generation for object-oriented software. *ESEC/FSE 2011*, 416-419.
- [4] Bagnall, A., Rayward-Smith, V., Whitley, I. (2001). The next release problem. *Information and Software Technology*, 43(14), 883-890.
- [5] O'Keeffe, M., O Cinnéide, M. (2008). Search-based refactoring for software maintenance. *Journal of Systems and Software*, 81(4), 502-516.
- [6] Zhou, Y., et al. (2022). Large language models are human-level prompt engineers. *arXiv:2211.01910*.
- [7] Fernando, C., et al. (2023). PromptBreeder: Self-referential self-improvement via prompt evolution. *arXiv:2309.16797*.
- [8] Romano, J., Kromrey, J. D., Coraggio, J., Skowronek, J. (2006). Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys? *Annual Meeting of the Florida Association of Institutional Research*.
- [9] Harman, M. (2010). The relationship between SBSE and emergent properties. *ICSE 2010*, 213-214.